

软件安全综合实践

实验二：缓冲区溢出漏洞利用

学院名称 智能与计算学部

专 业 网络空间安全

学生姓名 b1qx

年 班 级 2024 级

2026 年 6 月 25 日

1. 实验目的

- ◆ 理解函数调用过程中栈帧、EBP 和返回地址的组织方式。
- ◆ 理解数组越界写入导致缓冲区溢出的基本原理。
- ◆ 通过覆盖函数返回地址，观察程序控制流被重定向到指定函数的过程。
- ◆ 掌握利用 VC++ 6.0 调试器查看寄存器、内存、变量地址和反汇编指令的方法。

2. 过程记录

2.1 实验环境与程序功能

实验在 Windows XP 虚拟机中完成，使用 Microsoft Visual C++ 6.0 建立 Win32 控制台工程，并以 Debug 模式编译运行。程序包含 main、f 和 why_here 三个函数，其中 main 仅调用 f，源代码中没有正常调用 why_here 的语句。f 函数声明了仅含一个 int 元素的数组 b，却对 b[2] 进行赋值，由此产生数组越界写入。

```
#include <stdio.h>

void why_here()
{
    printf("why are you here?\n");
}

void f()
{
    int b[1];
    b[2] = (int)why_here;
}

int main(int argc, char *argv[])
{
    f();
    return 0;
}
```

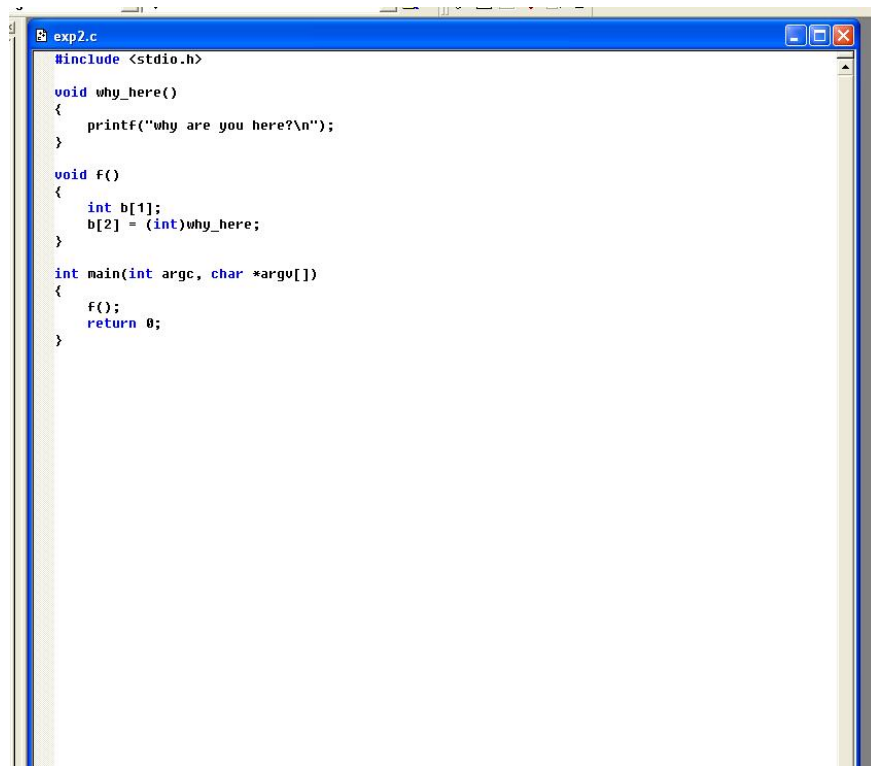


图 1 在 VC++ 6.0 中编写实验程序

2.2 程序编译与直接运行

程序在 Debug 配置下编译成功。直接运行后，控制台输出“why are you here?”，说明程序虽然没有显式调用 why_here 函数，却实际执行了该函数。随后系统提示 exp2.exe 发生错误并需要关闭。该异常并非实验失败，而是控制流被重定向后，why_here 执行结束时无法从栈中取得合法返回地址所产生的预期现象。

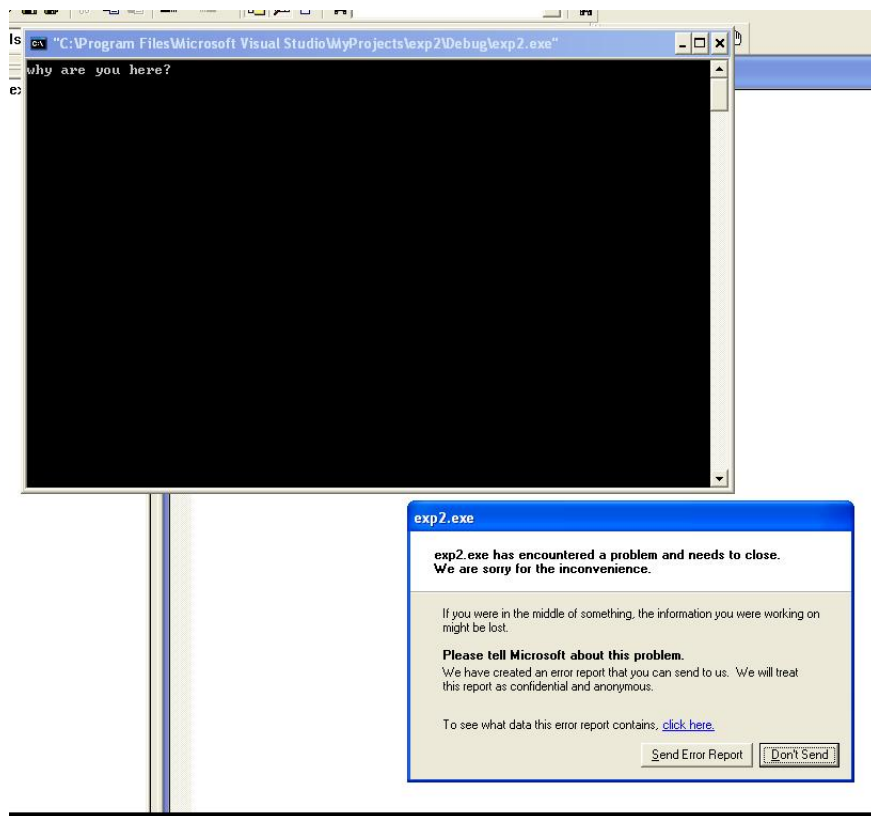


图 2 程序输出指定字符串后发生异常退出

2.3 在越界赋值语句处设置断点

在语句 `b[2] = (int)why_here` 处设置断点并启动调试。程序停止后，打开 Registers、Memory 和 Watch 窗口，查看 EBP 寄存器以及数组各元素的地址。调试结果如下：

```
EBP          = 0x0012FF2C
&b[0]        = 0x0012FF28
&b[1]        = 0x0012FF2C
&b[2]        = 0x0012FF30
&why_here    = 0x00401030
```

由地址关系可知，`&b[0]`等于 `EBP-4`，`&b[1]`等于 `EBP`，`&b[2]`等于 `EBP+4`。在 32 位 x86 函数栈帧中，EBP 位置保存调用者原来的 EBP，EBP+4 位置保存当前函数执行完毕后应返回到的地址。因此，数组越界访问 `b[2]`恰好定位到 f 函数保存的返回地址。

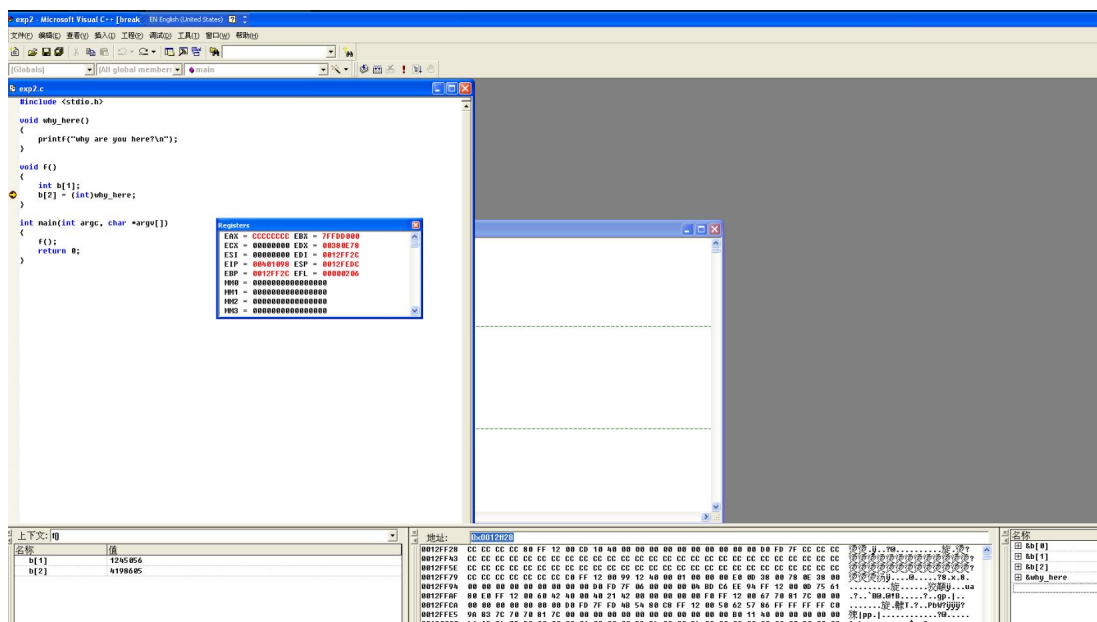


图 3 断点处观察 EBP、数组地址及内存内容

2.4 执行越界写入并覆盖返回地址

单步执行赋值语句后，反汇编窗口显示该 C 语句对应的关键指令为：

```
mov dword ptr [ebp+4], offset why_here (00401030)
```

该指令将 why_here 函数入口地址 0x00401030 写入[EBP+4]。由于[EBP+4]正是 f 函数保存返回地址的位置，因此 f 函数原来的正常返回地址被覆盖。此时黄色箭头已经移动到赋值语句之后，说明越界写入已经实际执行。

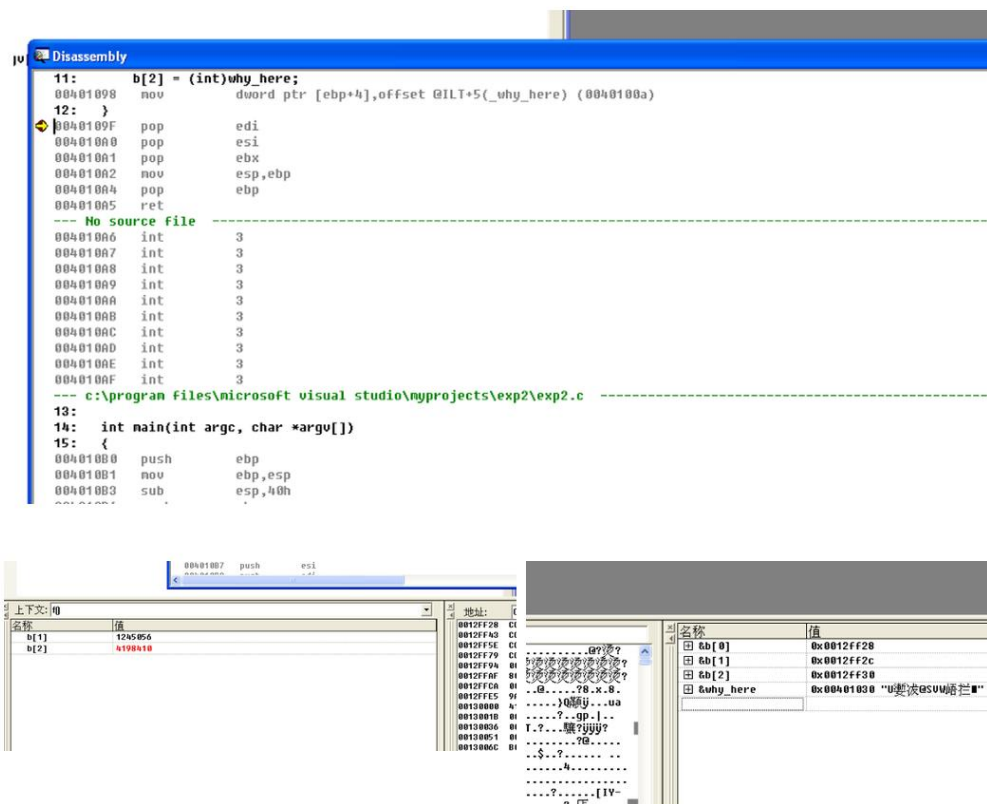


图 4 单步执行后返回地址被改写为 why_here 入口地址

2.5 观察控制流跳转

继续单步执行 f 函数结尾的栈恢复和 ret 指令。ret 会从栈顶取出返回地址并跳转，但该地址已被改写为 0x00401030，因此程序没有返回 main，而是进入 why_here 函数。调试器中的黄色执行箭头停在 printf 语句处，反汇编窗口也显示正在执行 why_here。此时 Watch 窗口无法继续按变量名访问 b，是因为 b 属于 f 函数的局部变量，程序已离开 f 的有效作用域。

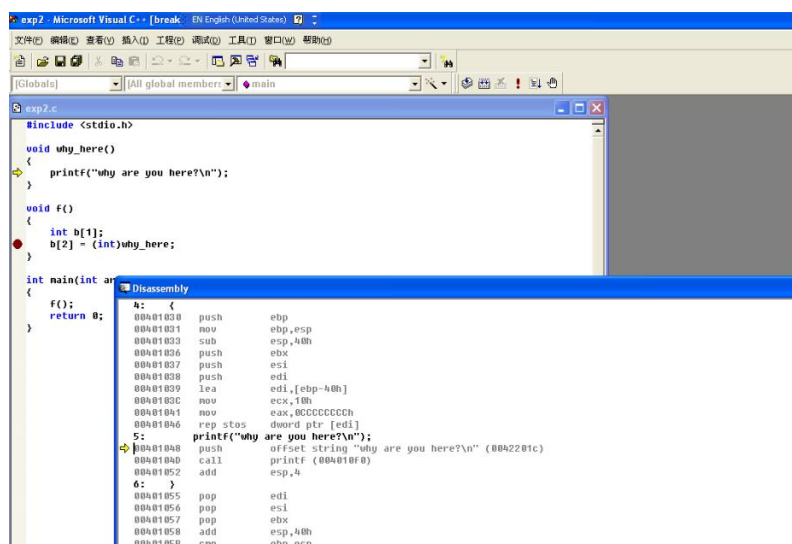


图 5 f 函数返回后程序跳入 why_here 函数

2.6 实验原理与结果分析

本实验利用的是典型的栈缓冲区越界写入。数组 b 仅分配了一个 int 元素，合法下标只有 0；b[2] 相对于数组首地址向高地址偏移 8 字节。在本次 VC++ 6.0 Debug 编译结果中，编译器将 b[0] 放置在 EBP-4 处，因此 b[2] 正好落在 EBP+4 处。栈帧关系可表示为：

高地址		
EBP + 4:	f 函数返回地址	← &b[2]
EBP :	保存的旧 EBP	← &b[1]
EBP - 4:	数组元素 b[0]	← &b[0]
低地址		

执行 b[2] = (int)why_here 后，f 函数的返回地址变为 why_here 入口地址。f 执行 ret 时，CPU 按照被篡改的地址转移控制权，因此打印出“why are you here? ”。但是该跳转不是由正常的 call why_here 指令完成的，栈中没有为 why_here 准备正确的返回现场。当 why_here 执行 ret 时，会把栈中的其他数据误当作返回地址，最终跳向无效位置并触发程序异常。

3. 心得体会

通过本次实验，我直观理解了缓冲区溢出的实质：越界写入可能破坏栈中的返回地址，从而改变程序控制流。调试结果中，`&b[2]`与 `EBP+4` 相同，反汇编指令也证明 `why_here` 的地址被写入返回地址位置。通过观察寄存器、内存和反汇编，我掌握了 C 语言代码与栈帧结构的对应关系，并理解了程序输出后崩溃的原因。今后编程时应严格检查数组下标和输入长度，并合理启用栈保护等安全机制。